

PROJECT TITLE

Briefly: A Private, Local AI Briefing Station That Tells You Only What You Want to Hear

STUDENT/TEAM INFORMATION

Team Name if any: Team # on Canvas:	Briefly 1
Team member 1 (Team Lead):	Vazrala, Bala – 33516100 bvazrala@uci.edu 
Team member 2:	Darmawan, Justin – 35518331 jdarmaw1@uci.edu 

ABSTRACT

Checking the weather, the calendar, and the news takes about ten seconds. Doing it on a phone takes ten minutes, because every unlock is an invitation to notifications and feeds. Voice assistants were supposed to close that gap, but they ask the owner to accept an always on microphone that streams audio to a company server.

Briefly is a desk station that shows only what its owner asked for. A LILYGO TTGO ESP32 renders short cards on an OLED and its built in LCD, rings an alarm, and snoozes when the user knocks on the case. A Python gateway on a laptop does the thinking. It runs Gemma 4 locally through Ollama, so no request leaves the local network.

The gateway hands the model a registry of tools rather than a fixed list of sources. Gemma replies with a tool name and a set of arguments. The gateway validates that reply against a

schema and builds every request itself, so the model never writes a URL. A user can name a team or a ticker we never anticipated and still get a live answer.

Every event the device publishes also travels to Azure IoT Hub over HTTPS. Message routing archives the raw events as JSON in Blob Storage. An Azure Function aggregates them into one row per day in Table Storage and serves those rows over HTTP to the dashboard, so the analytics run in the cloud rather than on the laptop.

The finished system meets every course requirement. It reads three sensors, drives two displays, communicates over Wi-Fi and MQTT, forwards telemetry to a cloud service, and computes analytics in that cloud. A card for a topic the user has never asked about before takes about twelve seconds, and returns from cache after that.

INTRODUCTION

Motivation/Background

Most mornings begin with the same ritual. Someone picks up a phone to check the weather, the calendar, and the news before walking out the door. The information itself takes seconds. The phone takes far longer, because opening it exposes the user to everything else competing for attention. The friction is small, but it repeats every single day.

Voice assistants were built to solve exactly this, and many households still refuse them. The reason is usually privacy. An always on microphone that sends audio to a company server is a trade many people will not make, and vendors have added subscription and advertising pressure on top of it. Ambient displays such as LaMetric avoid the microphone, but they are configured through rigid menus and still depend on vendor cloud services.

That leaves a gap. The information a person needs each morning is either one distracting phone unlock away, or it arrives through hardware they do not fully trust. We wanted the middle path: a small device on a desk or a nightstand that shows the weather, the next calendar event, headlines, and any custom topic its owner names, with personalization expressed in plain English rather than through menus.

This matters beyond convenience. Open weight models such as Gemma 4 and local runtimes such as Ollama have reached the point where assistant grade language understanding runs on a student laptop. A device that proves the concept on standard course hardware shows that private ambient computing no longer requires a vendor cloud.

Project Goals

G1, a standalone device. An ESP32 keeps time over NTP, stores alarms in on chip flash, rings a piezo buzzer, and snoozes on a double knock detected through the IMU tap interrupt. All of this continues to work when every other part of the system is offline.

G2, a glanceable briefing. Personalized cards covering weather, the next calendar event, headlines, and custom topics render across the TTGO color LCD and the OLED, and refresh on demand.

G3, natural language control. The owner sets preferences by typing plain English into a dashboard chat and refreshes everything by typing update. Gemma 4 runs locally through Ollama and handles preference parsing and card writing, while the update keyword is handled in code so the most common command never depends on model behavior.

G4, a cloud pipeline. Device events and telemetry reach Azure IoT Hub, and an Azure Function computes daily aggregates that drive a usage and climate dashboard.

G5, a stretch goal. Push to talk voice input using an I2S microphone, isolated in the architecture so the core system never depends on it.

Assumptions

- We assume one device and one owner. Nothing in the design accounts for several people sharing a station or for several stations sharing a gateway.
- We assume the laptop and the device sit on the same local network, reached through a phone hotspot or a home router. Campus Wi-Fi isolates clients from each other, so it cannot carry MQTT between the two halves.
- We assume the laptop is awake whenever a fresh briefing is wanted. The clock, the alarm, and the last set of cards survive without it, but new content does not.
- We assume a machine capable of holding a 12B model in memory. Our timings come from an M4 MacBook Air, and a weaker host would change them.
- We assume the free data sources we query stay available and unauthenticated, and that a briefing refreshed on demand is enough. Nothing in the design requires second by second updates.

SYSTEM ARCHITECTURE

Briefly is a three tier system. An ESP32 node renders cards and rings alarms. A Python gateway on a laptop does the thinking. Azure stores events and computes daily aggregates.

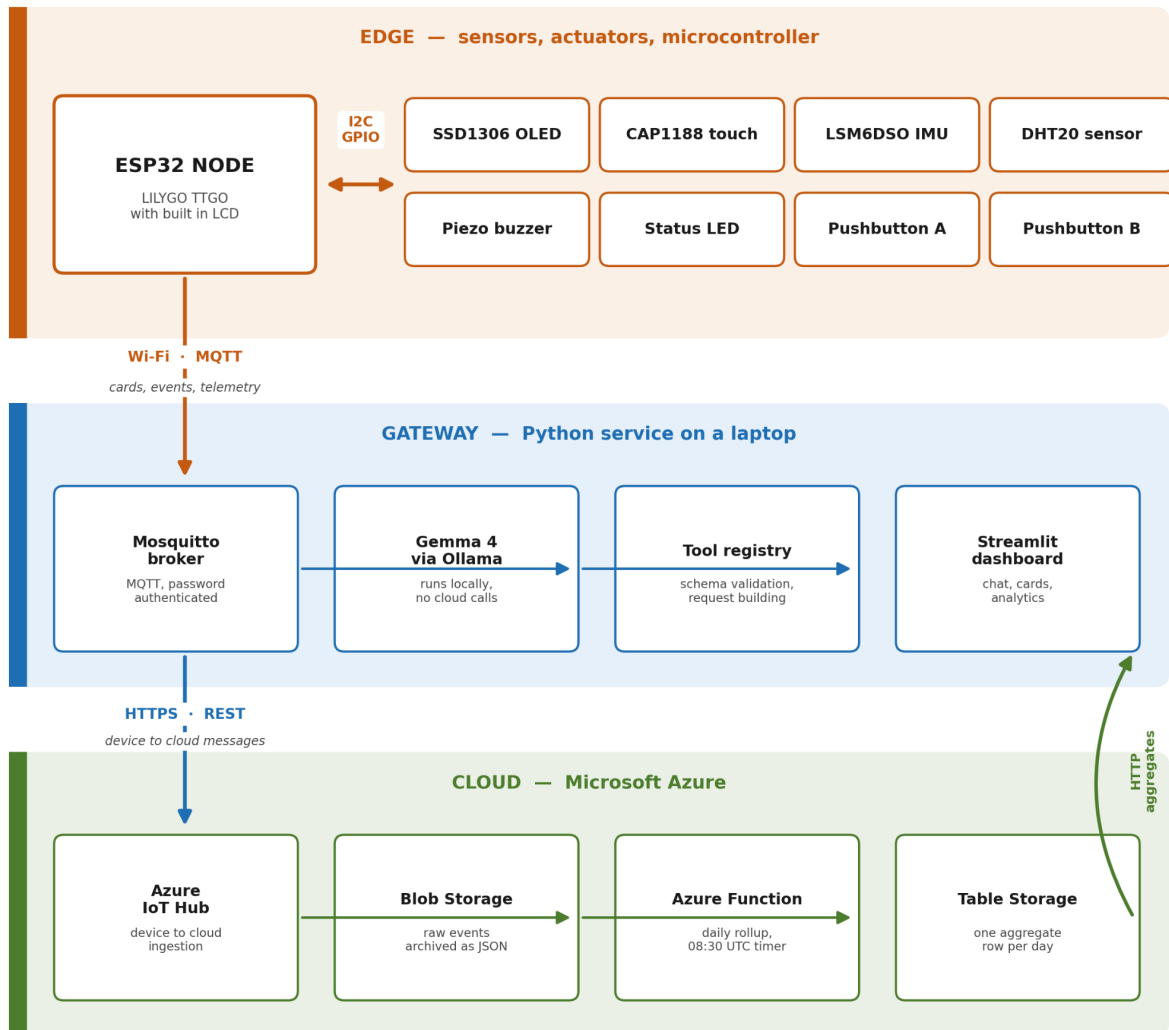
The node is a LILYGO TTGO ESP32 with a built in color LCD. Four peripherals share one I2C bus: an SSD1306 OLED, a CAP1188 capacitive touch breakout wired to copper tape pads, an LSM6DSO accelerometer, and a DHT20 temperature and humidity sensor. A piezo buzzer, a status LED, and two pushbuttons complete the board. Firmware is C++ under the Arduino framework, organized as FreeRTOS tasks for networking, display, input, and timekeeping, with alarms persisted in NVS flash.

The node reaches the gateway over Wi-Fi through a phone hotspot. The two exchange MQTT messages through a local Mosquitto broker protected by a username and password. The gateway publishes each card set as a retained message, so a device that reboots gets its display back the moment it reconnects. The device publishes knocks, key presses, and alarm actions, along with a temperature and humidity reading once a minute.

The gateway runs Gemma 4 locally through Ollama and exposes a registry of tools rather than a fixed source list. Gemma returns a tool name and arguments, the gateway validates that reply against a schema, and the gateway builds the request. A Streamlit dashboard provides the chat, the card view, and the analytics view.

The gateway forwards every event to Azure IoT Hub over HTTPS. Message routing writes the raw JSON into Blob Storage, and an Azure Function aggregates those blobs into one row per day in Table Storage. A second HTTP triggered function serves the most recent rows back to the dashboard.

Block diagram:



FINAL LIST OF HARDWARE COMPONENTS

Component/part	Quantity
LILYGO TTGO ESP32 board with built in LCD	1
SSD1306 OLED display 128x64	1
CAP1188 capacitive touch breakout with copper tape pads	1
LSM6DSO accelerometer and gyroscope	1
DHT20 temperature and humidity sensor	1

Component/part	Quantity
Piezo buzzer	1
Status LED with resistor	1
Pushbuttons	2
Breadboard, jumper wires, headers	1 set
Host laptop running the gateway and the local Gemma 4 model	1

PROJECT IMPLEMENTATION

Tasks/Milestones Completed

Task Completed	Date	Team Member
Component procurement and kit inventory	Jul 16	Justin
Repository structure and project scaffolding	Jul 30	Bala
Firmware: FreeRTOS tasks, NTP clock, NVS alarm storage	Aug 4	Bala
Mosquitto broker with password authentication	Aug 11	Bala
Gateway service publishing retained cards over MQTT	Aug 11	Bala
Virtual device simulator for testing without hardware	Aug 11	Bala
Tool registry with schema validation and result caching	Aug 11	Bala
Gemma 4 running locally through Ollama	Aug 11	Bala
Live sources: weather, web search, stock and crypto quotes, RSS	Aug 11	Bala
Streamlit dashboard with cards, chat, and settings	Aug 11	Bala
Hardware assembly, soldering, and I2C bus wiring	Aug 14	Justin
Alarm and double knock snooze verified on hardware	Aug 15	Justin
Enclosure and cable management	Aug 24	Justin
Azure IoT Hub ingestion over REST	Aug 25	Bala
IoT Hub message routing into Blob Storage	Aug 25	Bala
Azure Function computing daily aggregates in Table Storage	Aug 25	Bala
Cloud analytics view in the dashboard	Aug 26	Justin
Final report and demo video	Aug 28	Both

Bala built the firmware, the gateway, the tool registry, the dashboard, and the entire Azure pipeline. Justin handled procurement, assembled and soldered the board, wired the shared I2C

bus, verified the alarm and knock snooze on real hardware, built the enclosure, and added the cloud analytics view to the dashboard.

Two decisions kept the schedule intact. The first was a device simulator that speaks the same MQTT protocol as the firmware, which let the software progress while the board was still on the bench and let either of us test the full path alone. The second was firmware that skips any peripheral it cannot find and keeps running, so the four I2C devices could be brought up one at a time instead of debugged together.

The system was feature complete on August 25, when a manual run of the daily rollup aggregated 92 device records into a single day row and served them back over HTTP. That closed the last course requirement.

Challenges/Roadblocks

A dependency conflict blocked the cloud tier. The Azure IoT device SDK requires version 1 of the paho MQTT client library, and our gateway is written against version 2. The two cannot share one environment. We dropped the SDK and called the IoT Hub REST endpoint directly, which needs no extra package and removed the conflict permanently.

The model reasoned out loud. Gemma 4 wrote an internal monologue before answering, which is wasted effort when the output is a small JSON object, and every call ran past our thirty second limit. We disabled the monologue, held the weights in memory between calls, capped reply length, and cached each routing decision. A request that used to time out now finishes in about five seconds.

The first working cards were accurate and useless. Asked about the Steelers, the model returned the year the franchise was founded, because our prompt never said to prefer recent information. The prompt now bans background facts by name and receives the current date. A related problem appeared with SpaceX stock, where the model reported a planned IPO that had already happened after its weights were frozen. Ticker resolution now runs through a lookup table in code before the tool executes. Correctness that depends on recent facts belongs in code rather than in a prompt.

The cloud aggregates were nearly meaningless on the first run. Our telemetry heartbeat fires once a minute, so it produces roughly 1400 records a day and swamped the knocks and alarm actions we actually wanted to chart. The rollup now excludes telemetry from the event count while still using it for the temperature and humidity averages.

Tasks Not Completed

Task	Reason
Push to talk voice input	Listed in the proposal as a stretch goal that the core system would not depend on. Adding an I2S microphone, an amplifier, and local speech models needed more schedule than remained once the cloud tier was underway, so we protected the required work instead.

Task	Reason
Card quality comparison across the 4B and 12B models	Planned as a small evaluation of summarization reliability across a week of use. The Azure tier took longer than budgeted, and finishing the required cloud analytics was worth more than the measurement.

Every requirement in the proposal was met. Both unfinished items were optional: one was labelled a stretch goal from the start, and the other was an evaluation rather than a feature. Neither affects what the device does.

WEAK POINTS / FUTURE WORK

Latency. A topic the user has never asked about costs about twelve seconds on its first pass. Caching the routing decision helps, but the card writing call still runs on every refresh. A small model could handle routing while the 12B writes the text, and interests could resolve in parallel rather than one after another.

Search quality. Our web search returns general pages rather than fresh reporting, so the model sometimes works from thin material. A dedicated news endpoint would raise card quality more than further prompt tuning.

Cloud storage volume. IoT Hub writes a blob every sixty seconds whether or not any message arrived, which produces roughly 1400 files a day. A longer batch window or a lifecycle rule would cut that sharply. The rollup also buckets by UTC date, so events generated in the evening in California land in the following day.

Voice input. A small I2S microphone would let the owner hold a touch pad and ask a question out loud. The architecture already isolates this as a single adapter.

FINAL DEMO VIDEO

https://drive.google.com/file/d/1Z0ec_zqO--vBOlwzlSbYAmYhTDKT2mbl/view

SOURCE CODE

<https://github.com/bvazrala/Briefly>